Funciones y registros (struct) en C++

Resumen en mis palabras. El código completo está en funciones_y_structs.cpp

1. Funciones

Una función es un bloque de código con nombre que hace una tarea y se puede **reutilizar**. Tiene tipo de retorno, nombre y parámetros. Solo corre cuando alguien la llama (normalmente desde `main`).

```
int cuadrado(int x) {      // devuelve un int, recibe un int
    return x * x;
}

int r = cuadrado(4);       // r vale 16
```

Si no devuelve nada, el tipo es `void`. Simulación de `cuadrado(4)`: x toma el valor 4, calcula 4*4 = 16 y `return` lo manda de vuelta a donde se llamó.

2. Por valor vs por referencia

Esto es lo que más me confundía. Por defecto la función recibe una **copia**, así que no puede cambiar la variable original. Con & recibe el original.

```
void subirEdad(int& edad) {   // con &: modifica el original
    edad++;
}

int e = 20;
subirEdad(e);                 // ahora e vale 21
```

Forma	Qué hace	Cuándo la uso
`int x`	copia	datos pequeños que no quiero cambiar
`int& x`	usa el original y lo puede modificar	cuando la función debe cambiar algo
`const T& x`	usa el original pero no lo puede modificar	datos grandes que solo leo (vectores, structs)

3. Struct (registro)

Un struct agrupa **datos de distintos tipos** bajo un solo nombre. Un vector guarda muchos datos del mismo tipo; un struct guarda los datos de una sola "cosa": un estudiante, un producto, un libro...

```
struct Estudiante {
    string nombre;
    int edad;
    vector<double> notas;
};                            // ojo: lleva punto y coma al final
```

Cada dato dentro se llama **campo** y se accede con un punto:

```
Estudiante ana;
ana.nombre = "Ana";
```

```
ana.edad = 19;
ana.notas.push_back(4.0);

Estudiante sofia = {"Sofia", 20, {3.5, 4.0, 3.0}};   // forma corta
```

4. Structs y funciones juntos

Las funciones reciben structs igual que cualquier otro dato. Aplico la misma regla de antes: `const &` para leer, `&` para modificar.

```
double promedio(const Estudiante& e) {        // solo lee
    if (e.notas.empty()) return 0;
    double suma = 0;
    for (double n : e.notas) suma += n;
    return suma / e.notas.size();
}

void agregarNota(Estudiante& e, double nota) { // modifica
    e.notas.push_back(nota);
}
```

Simulación: Ana con notas {4.0, 5.0, 3.0}:

nota n	suma
4.0	4.0
5.0	9.0
3.0	12.0
fin	12.0 / 3 = **4.0**

5. Vector de structs

Lo más útil: una lista de registros, como una tabla donde cada fila es un struct.

```
vector<Estudiante> curso;
curso.push_back(ana);
curso.push_back(luis);

for (const Estudiante& e : curso) imprimir(e);

int mejorEstudiante(const vector<Estudiante>& lista) {
    int mejor = 0;
    for (size_t i = 1; i < lista.size(); i++)
        if (promedio(lista[i]) > promedio(lista[mejor])) mejor = i;
    return mejor;
}
```

El truco es guardar la **posición** del mejor y compararlo con cada siguiente. Con promedios Ana 4.0, Luis 4.75 y Sofia 3.5:

i	promedio de lista[i]	promedio de lista[mejor]	mejor
inicio	-	-	0 (Ana)

1	4.75 (Luis)	4.0 (Ana)	1 (Luis)
2	3.5 (Sofia)	4.75 (Luis)	1 (Luis)

6. Errores típicos

Error	Solución
Olvidar el `;` al cerrar el struct	Siempre `};`
Cambiar un struct y que no se guarde	Pasarlo con `&`, no por valor
Escribir `Estudiante.nombre`	Se usa la variable: `ana.nombre`
Promediar sin notas (dividir entre 0)	Revisar `notas.empty()` antes
Copiar un struct grande a cada función	Usar `const T&`

Conclusión

Las funciones me permiten dividir el programa en partes pequeñas que puedo probar por separado, y los structs me permiten tratar los datos de una cosa como un solo paquete. Juntos dan programas más ordenados: un vector de structs como "tabla" y funciones que la recorren. Lo que más tengo que cuidar es elegir bien cómo paso los parámetros (valor, `&` o `const &`) según si la función solo lee o también modifica.